

前端 WebSocket 對接規格書

Texas Hold'em 即時牌桌通訊協議、事件格式、狀態同步規則
與完整測試流程。

TEST ENDPOINT

wss://texaswss.test336.net

PROTOCOL

**WebSocket / JSON
Envelope**

VERSION

v3.1

UPDATED

2026-05-08

文件摘要

對接目標

提供前端實作即時牌桌、回放、報表與測試驗收所需的完整通訊契約。

資料來源

本文描述前端可觀察與可使用的訊息格式，不暴露服務內部實作細節。

驗收重點

以 `table_state` 作為權威狀態，並依 `action_request`、`award`、`hand_end` 驅動畫面流程。

章節索引

1. 連線方式

1.2 通訊格式

1.4 心跳

3. Server -> Client 訊息總表

4.1 Card

4.3 Table

4.5 Action

4.7 Player Result

5. Client -> Server 詳細格式

5.2 `register`

5.4 `auth_token`

5.6 `enter_game`

5.8 `player_action`

1.1 測試環境連線資訊

1.3 錯誤格式

2. Client -> Server 訊息總表

4. 共用資料格式

4.2 Round

4.4 Player

4.6 Award

4.8 Rake

5.1 `login`

5.3 `guest_login`

5.5 `enter_lobby`

5.7 `join_stakes`

5.9 `leave_room`

5.10 `switch_room`

5.12 `update_profile`

5.14 `hand_replay`

5.16 `my_daily_settlement_14d`

5.18 Partner 訊息

6.1 `login_ok`

6.3 `logout_ok`

6.5 `update_profile_ok`

6.7 `game_lobby_state`

6.9 `table_player_joined`

6.11 `wallet_state`

6.13 `hand_start`

6.15 `deal_card`

6.17 `hole_cards`

6.19 `action_request`

6.21 `player_action`

6.23 `deal_community`

6.25 `award`

6.27 `rebuy_offer`

6.29 `rebuy_result`

6.31 `my_hand_reports_ok`

5.11 `logout`

5.13 `rebuy_decision`

5.15 `my_hand_reports`

5.17 `ping`

6. Server -> Client 詳細格式

6.2 `auth_ok`

6.4 `forced_logout`

6.6 `lobby_state`

6.8 `table_joined`

6.10 `table_state`

6.12 `table_countdown`

6.14 `post_blinds`

6.16 `deal_private`

6.18 `betting_start`

6.20 `turn`

6.22 `betting_complete`

6.24 `showdown`

6.26 `hand_end`

6.28 `rebuy_ack`

6.30 `hand_replay_ok`

6.32 `my_daily_settlement_14d_ok`

6.33 Partner 回應

7.1 一般 showdown

7.3 All-in 後發完公共牌

8. 前端實作規則

10. 完整測試流程

10.2 登入與恢復登入

10.4 標準一手牌

10.6 Timeout 與斷線

10.8 All fold、未跟注金額、抽水

10.10 補碼

10.12 報表

11. 前端最小實作清單

7. 牌局事件順序

7.2 All fold

7.4 重連或刷新

9. 錯誤代碼

10.1 連線與心跳

10.3 大廳與入桌

10.5 操作合法性

10.7 離桌與換桌

10.9 Showdown、分池、抽水

10.11 牌局回放

10.13 Partner 流程

1. 連線方式

1.1 測試環境連線資訊

前端測試時，請連到以下 WebSocket URL：

TEXT

```
wss://texaswss.test336.net
```

正式或 HTTPS 環境可能使用：

TEXT

```
wss://<正式網域>
```

前端建議把 WebSocket URL 做成環境設定，例如：

TEXT

```
VITE_WS_URL=wss://texaswss.test336.net
```

注意：

- 前端不需要知道服務如何啟動。
- 前端不需要寫死 localhost。
- 同一份前端程式應能透過環境設定切換測試機、正式機。

1.2 通訊格式

所有訊息都是 JSON object。

Client -> Server：

JSON

```
{  
  "type": "message_type",  
  "data": {}  
}
```

Server -> Client :

JSON

```
{
  "type": "message_type",
  "data": {}
}
```

規則：

- `type` 必定是字串。
- `data` 通常是 object；若無資料可送 `{}`。
- 前端收到未知 `type` 時不要中斷連線，應記錄 log 並忽略。
- 前端解析 `bets`、`reveals` 這類以座位為 key 的 object 時，key 會是字串，例如 `"0"`、`"1"`。
- 金額大多是整數籌碼；報表或回放中可能是字串小數，例如 `"250.000000"`。

1.3 錯誤格式

JSON

```
{
  "type": "error",
  "data": {
    "code": "invalid_action",
    "message": "操作不合法",
    "detail": {}
  }
}
```

欄位：

欄位	型別	必填	說明
<code>code</code>	string	是	錯誤代碼
<code>message</code>	string	是	可顯示給使用者或寫入 log 的文字
<code>detail</code>	object	否	額外除錯資料

1.4 心跳

前端可定期送：

JSON

```
{
  "type": "ping",
  "data": {
    "ts": 1778209150000
  }
}
```

服務回：

JSON

```
{
  "type": "pong",
  "data": {
    "ts": 1778209150000
  }
}
```

2. Client -> Server 訊息總表

type	需要登入	用途
login	否	帳密登入
register	否	註冊帳號
guest_login	否	遊客登入
auth_token	否	用既有 token 恢復登入
partner_auth	否	Partner 驗證
partner_create_member	否	Partner 建立會員
partner_login_link	否	Partner 取得登入連結
partner_wallet_deposit	否	Partner 錢包上分
partner_wallet_withdraw	否	Partner 錢包下分
partner_wallet_balance	否	Partner 查錢包
partner_hand_replay	否	Partner 查牌局回放
enter_lobby	是	進入大廳

type	需要登入	用途
enter_game	是	進入指定遊戲大廳
join_stakes	是	加入指定盲注桌池
player_action	是	玩家操作
leave_room	是	離開目前牌桌
switch_room	是	換桌
logout	是	登出
update_profile	是	更新暱稱或頭像
rebuy_decision	是	回覆補碼邀請
hand_replay	是	查自己可看的單手回放
my_hand_reports	是	查自己的牌局紀錄
my_daily_settlement_14d	是	查 14 日每日結算
ping	否	心跳

3. Server -> Client 訊息總表

type	推送對象	用途
error	單一連線	錯誤
pong	單一連線	心跳回應
login_ok	單一連線	登入成功
auth_ok	單一連線	token 恢復成功
logout_ok	單一連線	登出成功
forced_logout	單一連線	被迫登出
update_profile_ok	單一連線	更新個人資料成功
partner_auth_ok	單一連線	Partner 驗證成功
partner_member_ok	單一連線	Partner 會員建立成功

type	推送對象	用途
partner_login_link_ok	單一連線	Partner 登入連結
partner_wallet_deposit_ok	單一連線	Partner 上分成功
partner_wallet_withdraw_ok	單一連線	Partner 下分成功
partner_wallet_balance_ok	單一連線	Partner 錢包餘額
partner_hand_replay_ok	單一連線	Partner 牌局回放
lobby_state	單一連線	大廳狀態
game_lobby_state	單一連線	指定遊戲大廳狀態
table_joined	單一連線	入桌成功
table_player_joined	桌上玩家	有玩家入桌
table_state	單一或桌上玩家	牌桌完整狀態
wallet_state	單一連線	錢包與桌上籌碼
table_countdown	桌上玩家	開局倒數
hand_start	桌上玩家	新一手開始
post_blinds	桌上玩家	盲注已下
deal_card	桌上玩家	發一張背面手牌
deal_private	單一玩家	玩家自己的明牌資料
hole_cards	單一玩家	玩家自己的兩張手牌
betting_start	桌上玩家	某街下注開始
action_request	輪到的玩家	要求玩家操作
turn	桌上玩家	目前輪到誰
player_action	桌上玩家	玩家已操作
betting_complete	桌上玩家	某街下注結束
deal_community	桌上玩家	發公共牌
showdown	桌上玩家	攤牌
award	桌上玩家	派彩資料
hand_end	桌上玩家	一手結束

type	推送對象	用途
rebuy_offer	單一玩家	補碼邀請
rebuy_ack	單一玩家	補碼回覆已收到
rebuy_result	單一玩家	補碼結果
hand_replay_ok	單一玩家	牌局回放
my_hand_reports_ok	單一玩家	牌局紀錄
my_daily_settlement_14d_ok	單一玩家	14 日每日結算

4. 共用資料格式

4.1 Card

牌用兩碼字串表示。

TEXT

As, Ah, Ad, Ac
Ks, Kh, Kd, Kc
...
2s, 2h, 2d, 2c

第一碼：

值	說明
A	Ace
K	King
Q	Queen
J	Jack
T	Ten
9~2	點數

第二碼：

値	説明
s	spade
h	heart
d	diamond
c	club

4.2 Round

TEXT

waiting
preflop
flop
turn
river

4.3 Table

JSON

```
{
  "table_id": "texas_holdem_t25_50_xxxxxxxx",
  "game_id": "texas_holdem",
  "stakes_id": "t25_50",
  "small_blind": 25,
  "big_blind": 50,
  "min_buyin": 1000,
  "max_buyin": 20000,
  "max_players": 6,
  "status": "playing",
  "round": "flop",
  "hand_id": 709,
  "community": ["Kh", "5c", "3d"],
  "pot": 750,
  "current_bet": 200,
  "bets": {
    "0": 200,
    "1": 200,
    "2": 200
  },
  "round_total_bet": 600,
  "dealer_seat": 1,
  "sb_seat": 2,
  "bb_seat": 0,
  "players": []
}
```

欄位：

欄位	型別	說明
<code>table_id</code>	string	牌桌 ID
<code>game_id</code>	string	遊戲 ID，目前為 <code>texas_holdem</code>
<code>stakes_id</code>	string	盲注級別
<code>small_blind</code>	number	小盲
<code>big_blind</code>	number	大盲
<code>min_buyin</code>	number	最小帶入
<code>max_buyin</code>	number	最大帶入
<code>max_players</code>	number	最大座位數
<code>status</code>	string	<code>waiting</code> 或 <code>playing</code>
<code>round</code>	string	目前街別
<code>hand_id</code>	number	目前手牌 ID
<code>community</code>	string[]	公共牌
<code>pot</code>	number	目前總底池
<code>current_bet</code>	number	本街目前最高下注額
<code>bets</code>	object	本街每個座位目前桌上下注
<code>round_total_bet</code>	number	本街目前總下注
<code>dealer_seat</code>	number/null	莊位
<code>sb_seat</code>	number/null	小盲座位
<code>bb_seat</code>	number/null	大盲座位
<code>players</code>	Player[]	玩家列表

重要規則：

- `pot` 是已累積的總底池。
- `bets` 是「目前街」桌面上可見下注，不是整手牌總貢獻。
- 進入新一街時，前一街 `bets` 應歸零。
- 已棄牌或離桌玩家不應在新一街殘留上一街 `bets`。
- `round_total_bet` 只表示目前街的下注總額。

- 一手牌總貢獻請看 `award.player_results[].contrib_amount` 或 `hand_end.player_results[].contrib_amount`。

4.4 Player

JSON

```
{
  "user_id": 361,
  "username": "小老鼠",
  "avatar": "avatar_001",
  "seat": 2,
  "chips": 19750,
  "bet": 200,
  "is_sitting_out": false,
  "is_bot": false,
  "is_disconnected": false,
  "in_hand": true,
  "hole_count": 2,
  "last_action": "bet",
  "last_action_at": 1778209130037
}
```

欄位：

欄位	型別	說明
<code>user_id</code>	number	使用者 ID
<code>username</code>	string	顯示名稱
<code>avatar</code>	string	頭像代碼
<code>seat</code>	number	座位
<code>chips</code>	number	玩家目前桌上可用籌碼
<code>bet</code>	number	玩家本街桌上下注
<code>is_sitting_out</code>	boolean	是否暫停
<code>is_bot</code>	boolean	是否機器人
<code>is_disconnected</code>	boolean	是否斷線
<code>in_hand</code>	boolean	是否仍在手牌中
<code>hole_count</code>	number	手牌張數，其他玩家通常只顯示數量
<code>last_action</code>	string/null	最近動作
<code>last_action_at</code>	number/null	最近動作時間戳

4.5 Action

前端送出的玩家操作：

TEXT

```
fold
check
call
bet
raise
allin
```

服務端可能回傳的 timeout 或系統動作：

TEXT

```
fold_timeout
check_timeout
```

4.6 Award

JSON

```
{
  "seat": 0,
  "amount": 765,
  "breakdown": [
    {
      "seat": 0,
      "amount": 665,
      "pot_index": 0,
      "hand_rank": "two_pair",
      "best5": ["Jd", "Ks", "Kh", "3d", "3s"]
    }
  ],
  "hand_rank": "two_pair",
  "best5": ["Jd", "Ks", "Kh", "3d", "3s"]
}
```

4.7 Player Result

JSON

```
{
  "seat": 0,
  "user_id": 1648,
  "username": "jczzgbnbjsuq",
  "avatar": "avatar_007",
  "hand_rank": "two_pair",
  "best5": ["Jd", "Ks", "Kh", "3d", "3s"],
  "hole_cards": ["Jd", "Ks"],
  "contrib_amount": 300,
  "win_amount": 765,
  "net_amount": 465
}
```

說明：

- **contrib_amount** 是玩家整手牌實際投入。
- **win_amount** 是玩家本手收到的派彩金額。
- **net_amount = win_amount - contrib_amount**。
- 玩家即使贏得底池，也可能因抽水導致 **net_amount** 為負。

4.8 Rake

JSON

```
{
  "pot_total": 850,
  "rake_percent": 0.1,
  "rake_cap_bb": 20,
  "rake_cap_amount": 1000,
  "rake_amount": 85,
  "payout_total": 765,
  "raked_pot_total": 850,
  "uncalled_return": 0,
  "no_flop_no_drop": false
}
```

規則：

- **pot_total** 是本手底池總額。
 - **rake_amount** 是抽水。
 - **payout_total** 是實際派彩金額。
 - 未被跟注的下注應退回玩家，不應抽水。
 - 若牌局在 flop 前結束，應符合 no-flop-no-drop，不抽水。
-

5. Client -> Server 詳細格式

5.1 login

JSON

```
{
  "type": "login",
  "data": {
    "username": "alice",
    "password": "password123"
  }
}
```

成功回 login_ok 。

5.2 register

JSON

```
{
  "type": "register",
  "data": {
    "username": "alice",
    "password": "password123",
    "avatar": "avatar_001"
  }
}
```

成功回 login_ok 。

5.3 guest_login

JSON

```
{
  "type": "guest_login",
  "data": {
    "device_id": "browser-device-id",
    "username": "Guest",
    "avatar": "avatar_001"
  }
}
```

成功回 login_ok 。

5.4 auth_token

JSON

```
{
  "type": "auth_token",
  "data": {
    "token": "jwt-or-session-token"
  }
}
```

成功回 `auth_ok` 。若玩家原本在桌上，可能接著收到 `table_joined` 、 `table_state` 、 `wallet_state` 。

5.5 enter_lobby

JSON

```
{
  "type": "enter_lobby",
  "data": {}
}
```

成功回 `lobby_state` 。

5.6 enter_game

JSON

```
{
  "type": "enter_game",
  "data": {
    "game_id": "texas_holdem"
  }
}
```

成功回 `game_lobby_state` 。

5.7 join_stakes

JSON

```
{
  "type": "join_stakes",
  "data": {
    "game_id": "texas_holdem",
    "stakes_id": "t25_50",
    "buyin": 2000
  }
}
```

可能回：

- `table_joined`
- `table_player_joined`
- `wallet_state`
- `table_countdown`
- `error`

5.8 `player_action`

JSON

```
{
  "type": "player_action",
  "data": {
    "action": "call"
  }
}
```

下注或加注：

JSON

```
{
  "type": "player_action",
  "data": {
    "action": "raise",
    "amount": 300
  }
}
```

欄位：

欄位	型別	必填	說明
<code>action</code>	string	是	<code>fold</code> 、 <code>check</code> 、 <code>call</code> 、 <code>bet</code> 、 <code>raise</code> 、 <code>allin</code>
<code>amount</code>	number	視動作	<code>bet</code> 、 <code>raise</code> 使用，表示本街目標下注額

前端必須以最近一次 `action_request.allowed` 判斷可用操作。

5.9 leave_room

JSON

```
{
  "type": "leave_room",
  "data": {}
}
```

若玩家還在手牌中，離桌可能會造成自動棄牌。可能回 `table_state`、`wallet_state`。

5.10 switch_room

JSON

```
{
  "type": "switch_room",
  "data": {
    "buyin": 4291
  }
}
```

可能回：

- 原桌 `table_state`
- 新桌 `table_joined`
- 新桌 `table_player_joined`
- `wallet_state`

5.11 logout

JSON

```
{
  "type": "logout",
  "data": {}
}
```

成功回 `logout_ok`。

5.12 `update_profile`

JSON

```
{
  "type": "update_profile",
  "data": {
    "username": "new_name",
    "avatar": "avatar_007"
  }
}
```

成功回 `update_profile_ok` 。

5.13 `rebuy_decision`

接受補碼：

JSON

```
{
  "type": "rebuy_decision",
  "data": {
    "accept": true,
    "amount": 2000
  }
}
```

拒絕補碼：

JSON

```
{
  "type": "rebuy_decision",
  "data": {
    "accept": false
  }
}
```

可能回 `rebuy_ack` 、 `rebuy_result` 、 `wallet_state` 。

5.14 `hand_replay`

JSON

```
{
  "type": "hand_replay",
  "data": {
    "hand_id": 709
  }
}
```

若同一 `hand_id` 可能跨桌重複，請帶 `table_id`：

JSON

```
{
  "type": "hand_replay",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709
  }
}
```

成功回 `hand_replay_ok`。

5.15 `my_hand_reports`

JSON

```
{
  "type": "my_hand_reports",
  "data": {
    "limit": 20,
    "offset": 0
  }
}
```

成功回 `my_hand_reports_ok`。

5.16 `my_daily_settlement_14d`

JSON

```
{
  "type": "my_daily_settlement_14d",
  "data": {}
}
```

成功回 `my_daily_settlement_14d_ok`。

5.17 `ping`

JSON

```
{
  "type": "ping",
  "data": {
    "ts": 1778209150000
  }
}
```

成功回 `pong` 。

5.18 Partner 訊息

Partner 類訊息通常由營運、後台或整合方使用，一般遊戲前端若不負責 Partner 流程可不實作。

`partner_auth`

JSON

```
{
  "type": "partner_auth",
  "data": {
    "partner_id": "partner_a",
    "secret": "secret"
  }
}
```

成功回 `partner_auth_ok` 。

`partner_create_member`

JSON

```
{
  "type": "partner_create_member",
  "data": {
    "partner_user_id": "user-001",
    "username": "alice",
    "avatar": "avatar_001"
  }
}
```

成功回 `partner_member_ok` 。

`partner_login_link`

JSON

```
{
  "type": "partner_login_link",
  "data": {
    "partner_user_id": "user-001"
  }
}
```

成功回 `partner_login_link_ok` 。

partner_wallet_deposit

JSON

```
{
  "type": "partner_wallet_deposit",
  "data": {
    "partner_user_id": "user-001",
    "amount": 1000,
    "tx_id": "deposit-001"
  }
}
```

成功回 `partner_wallet_deposit_ok` 。

partner_wallet_withdraw

JSON

```
{
  "type": "partner_wallet_withdraw",
  "data": {
    "partner_user_id": "user-001",
    "amount": 1000,
    "tx_id": "withdraw-001"
  }
}
```

成功回 `partner_wallet_withdraw_ok` 。

partner_wallet_balance

JSON

```
{
  "type": "partner_wallet_balance",
  "data": {
    "partner_user_id": "user-001"
  }
}
```

成功回 `partner_wallet_balance_ok` 。

partner_hand_replay

JSON

```
{
  "type": "partner_hand_replay",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709
  }
}
```

成功回 `partner_hand_replay_ok` 。

6. Server -> Client 詳細格式

6.1 login_ok

JSON

```
{
  "type": "login_ok",
  "data": {
    "token": "session-token",
    "user": {
      "user_id": 361,
      "username": "小老鼠",
      "avatar": "avatar_001"
    },
    "wallet_balance": 495684,
    "table_chips": 0
  }
}
```


6.2 auth_ok

JSON

```
{
  "type": "auth_ok",
  "data": {
    "user": {
      "user_id": 361,
      "username": "小老鼠",
      "avatar": "avatar_001"
    },
    "wallet_balance": 495684,
    "table_chips": 4291
  }
}
```

6.3 logout_ok

JSON

```
{
  "type": "logout_ok",
  "data": {}
}
```

6.4 forced_logout

JSON

```
{
  "type": "forced_logout",
  "data": {
    "reason": "login_elsewhere",
    "message": "帳號已在其他地方登入"
  }
}
```

收到後前端應清除登入狀態並回登入頁或提示重新登入。

6.5 update_profile_ok

JSON

```
{
  "type": "update_profile_ok",
  "data": {
    "user": {
      "user_id": 361,
      "username": "new_name",
      "avatar": "avatar_007"
    }
  }
}
```

6.6 lobby_state

JSON

```
{
  "type": "lobby_state",
  "data": {
    "games": [
      {
        "game_id": "texas_holdem",
        "name": "Texas Hold'em",
        "stakes": []
      }
    ]
  }
}
```

6.7 game_lobby_state

JSON

```
{
  "type": "game_lobby_state",
  "data": {
    "game_id": "texas_holdem",
    "stakes": [
      {
        "stakes_id": "t25_50",
        "small_blind": 25,
        "big_blind": 50,
        "min_buyin": 1000,
        "max_buyin": 20000,
        "players": 12,
        "tables": 4
      }
    ]
  }
}
```

6.8 table_joined

JSON

```
{
  "type": "table_joined",
  "data": {
    "hero_seat": 2,
    "table": {},
    "waiting_this_hand": false
  }
}
```

欄位：

欄位	型別	說明
hero_seat	number/null	自己的座位；旁觀或已離桌時可能為 null
table	Table	牌桌狀態
waiting_this_hand	boolean	是否需等下一手才能參與

6.9 table_player_joined

JSON

```
{
  "type": "table_player_joined",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "player": {
      "user_id": 1453,
      "username": "player",
      "avatar": "avatar_008",
      "seat": 3,
      "chips": 13674,
      "is_sitting_out": false,
      "is_bot": false,
      "in_hand": false
    }
  }
}
```

6.10 table_state

JSON

```
{
  "type": "table_state",
  "data": {
    "hero_seat": 2,
    "table": {}
  }
}
```

`table_state` 是前端同步牌桌畫面的權威狀態。若事件訊息與 `table_state` 有差異，前端應以最新 `table_state` 修正畫面。

6.11 wallet_state

JSON

```
{
  "type": "wallet_state",
  "data": {
    "wallet_balance": 495684,
    "table_chips": 4291
  }
}
```

6.12 table_countdown

JSON

```
{
  "type": "table_countdown",
  "data": {
    "table_id": "texas_holdem_t25_50_d584243b",
    "seconds": 3
  }
}
```

6.13 hand_start

JSON

```
{
  "type": "hand_start",
  "data": {
    "hand_id": 709,
    "dealer_seat": 1,
    "sb_seat": 2,
    "bb_seat": 0,
    "table": {}
  }
}
```

6.14 post_blinds

JSON

```
{
  "type": "post_blinds",
  "data": {
    "hand_id": 709,
    "table_id": "texas_holdem_t25_50_d222894d",
    "sb_seat": 2,
    "bb_seat": null,
    "sb_amount": 25,
    "bb_amount": null,
    "pot": 25,
    "bets": {
      "0": 0,
      "1": 0,
      "2": 25
    },
    "current_bet": 0,
    "round_total_bet": 25
  }
}
```

大盲會再收到一包 `post_blinds`，其中 `bb_seat` 與 `bb_amount` 有值。

6.15 deal_card

JSON

```
{
  "type": "deal_card",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709,
    "seat": 2,
    "card_index": 0
  }
}
```

不包含牌面。

6.16 deal_private

JSON

```
{
  "type": "deal_private",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709,
    "seat": 2,
    "card_index": 0,
    "card": "Ad"
  }
}
```

只會送給該玩家。

6.17 hole_cards

JSON

```
{
  "type": "hole_cards",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709,
    "cards": ["Ad", "8h"]
  }
}
```

只會送給該玩家。

6.18 betting_start

JSON

```
{
  "type": "betting_start",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709,
    "round": "flop",
    "action_seat": 2,
    "current_bet": 0,
    "pot": 150,
    "bets": {
      "0": 0,
      "1": 0,
      "2": 0
    }
  }
}
```

6.19 action_request

JSON

```
{
  "type": "action_request",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709,
    "seat": 2,
    "to_call": 50,
    "current_bet": 50,
    "my_bet": 0,
    "min_raise_to": 100,
    "big_blind": 50,
    "pot": 75,
    "round_total_bet": 75,
    "allowed": ["fold", "call", "raise", "allin"],
    "timeout": 10
  }
}
```

前端收到後才應顯示操作按鈕。若沒有收到 `action_request`，即使 `turn.seat` 是自己，也不要送操作。

6.20 turn

JSON

```
{
  "type": "turn",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709,
    "seat": 2,
    "timeout": 10,
    "round": "river"
  }
}
```

6.21 player_action

JSON

```
{
  "type": "player_action",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709,
    "seat": 2,
    "action": "bet",
    "paid": 200,
    "pot": 350,
    "current_bet": 200,
    "bets": {
      "0": 0,
      "1": 0,
      "2": 200
    },
    "round_total_bet": 200,
    "meta": {
      "reason": "leave_room"
    }
  }
}
```

`meta` 可能不存在。若 `meta.reason` 是 `leave_room`、`switch_room`、`logout`，通常代表玩家因離開相關操作被系統處理。

6.22 betting_complete

JSON

```
{
  "type": "betting_complete",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709,
    "pot": 850,
    "bets": {
      "0": 50,
      "1": 50,
      "2": 200
    },
    "current_bet": 50
  }
}
```

前端可用它作為準備收籌碼或進入下一階段的訊號，但畫面最終仍應以 `table_state` 為準。

6.23 deal_community

Flop :

JSON

```
{
  "type": "deal_community",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709,
    "round": "flop",
    "cards": ["Kh", "5c", "3d"],
    "community": ["Kh", "5c", "3d"]
  }
}
```

Turn 或 River :

JSON

```
{
  "type": "deal_community",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709,
    "round": "river",
    "cards": ["2h"],
    "community": ["Kh", "5c", "3d", "3s", "2h"]
  }
}
```

6.24 showdown

JSON

```
{
  "type": "showdown",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709,
    "reason": "showdown",
    "reveals": {
      "0": {
        "seat": 0,
        "hole": ["Jd", "Ks"],
        "best5": ["Jd", "Ks", "Kh", "3d", "3s"],
        "hand_rank": "two_pair"
      }
    }
  }
}
```

reason 可能是：

TEXT

```
showdown
all_fold
allin_showdown
```

若 **reason** 是 **all_fold**，**reveals** 可能是空 object。

6.25 award

JSON

```
{
  "type": "award",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709,
    "reason": "showdown",
    "awards": [],
    "award_breakdown": [],
    "player_results": [],
    "rake": {}
  }
}
```

前端應在此階段播放派彩動畫。注意：**award** 是派彩資料，不代表 **players[].chips** 已經完成最終同步；最終籌碼以 **hand_end.table.players[].chips** 或後續 **table_state** 為準。

6.26 hand_end

JSON

```
{
  "type": "hand_end",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "hand_id": 709,
    "reason": "showdown",
    "awards": [],
    "award_breakdown": [],
    "reveals": {},
    "player_results": [],
    "rake": {},
    "table": {}
  }
}
```

`hand_end.table.players[].chips` 是一手結束後的最終桌上籌碼。

6.27 rebuy_offer

JSON

```
{
  "type": "rebuy_offer",
  "data": {
    "table_id": "texas_holdem_t25_50_d222894d",
    "min_buyin": 1000,
    "max_buyin": 20000,
    "current_chips": 0,
    "wallet_balance": 495684
  }
}
```

6.28 rebuy_ack

JSON

```
{
  "type": "rebuy_ack",
  "data": {
    "accept": true,
    "amount": 2000
  }
}
```

6.29 rebuy_result

JSON

```
{
  "type": "rebuy_result",
  "data": {
    "ok": true,
    "amount": 2000,
    "wallet_balance": 493684,
    "table_chips": 2000
  }
}
```

6.30 hand_replay_ok

JSON

```
{
  "type": "hand_replay_ok",
  "data": {
    "ok": true,
    "hero_seat": 2,
    "hero_hole_cards": ["8h", "Ad"],
    "replay": {
      "game_code": "texas_holdem",
      "table_id": "texas_holdem_t25_50_41e16ff9",
      "hand_id": 630,
      "status": "FINISHED",
      "reason": "showdown",
      "pot_total": 850,
      "rake_amount": 85,
      "payout_total": 765,
      "community": [],
      "reveals": {},
      "awards": [],
      "award_breakdown": [],
      "rake": {},
      "table_snapshot": {},
      "players": [],
      "timeline": []
    }
  }
}
```

timeline 每筆格式：

JSON

```
{
  "event_no": 1,
  "event": "player_action",
  "round": "flop",
  "seat": 2,
  "user_id": 361,
  "event_at_ms": 1778209130037,
  "elapsed_ms": 17507,
  "payload": {}
}
```

回放注意事項：

- `timeline[].event` 會對應實戰中推送過的事件名稱。
- 前端播放回放時應依 `event_no` 或 `elapsed_ms` 排序。
- 若玩家已在某街 fold，下一街 `bets` 不應殘留該玩家上一街下注。
- 回放資料若遇到舊資料異常，前端可做防呆，但新資料應以本文件規格為準。

6.31 my_hand_reports_ok

JSON

```
{
  "type": "my_hand_reports_ok",
  "data": {
    "items": [
      {
        "table_id": "texas_holdem_t25_50_d222894d",
        "hand_id": 709,
        "started_at": 1778209112530,
        "ended_at": 1778209160835,
        "net_amount": 465,
        "contrib_amount": 300,
        "win_amount": 765
      }
    ],
    "total": 1,
    "limit": 20,
    "offset": 0
  }
}
```

6.32 my_daily_settlement_14d_ok

JSON

```
{
  "type": "my_daily_settlement_14d_ok",
  "data": {
    "items": [
      {
        "date": "2026-05-08",
        "hands": 12,
        "contrib_amount": 3000,
        "win_amount": 4200,
        "net_amount": 1200,
        "rake_amount": 300
      }
    ]
  }
}
```

6.33 Partner 回應

partner_auth_ok

JSON

```
{
  "type": "partner_auth_ok",
  "data": {
    "ok": true,
    "partner_id": "partner_a"
  }
}
```

partner_member_ok

JSON

```
{
  "type": "partner_member_ok",
  "data": {
    "ok": true,
    "partner_user_id": "user-001",
    "user_id": 361
  }
}
```

partner_login_link_ok

JSON

```
{
  "type": "partner_login_link_ok",
  "data": {
    "ok": true,
    "url": "https://example.com/login?token=..."
  }
}
```

partner_wallet_deposit_ok

JSON

```
{
  "type": "partner_wallet_deposit_ok",
  "data": {
    "ok": true,
    "partner_user_id": "user-001",
    "amount": 1000,
    "balance": 5000,
    "tx_id": "deposit-001"
  }
}
```

partner_wallet_withdraw_ok

JSON

```
{
  "type": "partner_wallet_withdraw_ok",
  "data": {
    "ok": true,
    "partner_user_id": "user-001",
    "amount": 1000,
    "balance": 4000,
    "tx_id": "withdraw-001"
  }
}
```

partner_wallet_balance_ok

JSON

```
{
  "type": "partner_wallet_balance_ok",
  "data": {
    "ok": true,
    "partner_user_id": "user-001",
    "balance": 4000
  }
}
```

partner_hand_replay_ok

JSON

```
{
  "type": "partner_hand_replay_ok",
  "data": {
    "ok": true,
    "replay": {}
  }
}
```

7. 牌局事件順序

7.1 一般 showdown

TEXT

```
table_countdown
hand_start
post_blinds
post_blinds
deal_card / deal_private / hole_cards
betting_start
action_request / turn
player_action
table_state
betting_complete
deal_community
table_state
betting_start
...
river betting_complete
table_state
showdown
award
hand_end
table_state
```

前端動畫建議：

- `deal_card` 播背面發牌。
- `deal_private` 或 `hole_cards` 顯示自己的手牌。
- `player_action` 更新玩家動作與桌上籌碼。
- `betting_complete` 準備收籌碼。
- 新街 `table_state.round` 改變時可播放收籌碼進底池。
- `showdown` 翻牌。
- `award` 播派彩。

- `hand_end` 同步最終籌碼。

7.2 All fold

TEXT

```
player_action: fold / fold_timeout  
showdown reason=all_fold  
award  
hand_end  
table_state
```

規則：

- 不一定有 `betting_complete`。
- 不一定有 `reveals`。
- 最後未棄牌玩家拿下可贏底池。
- 未被跟注金額應退回，不列入抽水。
- flop 前結束不抽水。

7.3 All-in 後發完公共牌

可能流程：

TEXT

```
player_action action=allin  
player_action action=call  
betting_complete  
deal_community  
deal_community  
showdown  
award  
hand_end
```

規則：

- 若所有仍在牌中的玩家都已 all-in，後續可能不再有 `action_request`。
- 服務會補齊公共牌後進入 showdown。

7.4 重連或刷新

前端重連後：

TEXT

```
auth_token
auth_ok
table_joined
table_state
wallet_state
```

可能只收到部分訊息，前端應以最新 `table_state` 重建畫面。

8. 前端實作規則

1. `table_state` 是權威狀態。
2. `action_request` 只會送給輪到操作的玩家。
3. `turn` 是廣播，給所有人顯示目前輪到誰。
4. `deal_card` 不包含牌面，只能用來播發背面牌。
5. 自己的手牌只信任 `deal_private` 與 `hole_cards`。
6. `bets` 只代表目前街桌上籌碼。
7. 新一街開始時，前一街 `bets` 應全部歸零。
8. 已 fold 或已離桌玩家，新一街不應保留上一街下注。
9. `award` 是派彩資料，`hand_end.table.players[].chips` 是最終籌碼。
10. all-fold 的未跟注金額不抽水。
11. flop 前結束不抽水。
12. 收到 `forced_logout` 要清除登入狀態。
13. 前端送操作前必須確認最近一次 `action_request.hand_id`、`seat`、`allowed` 都符合目前狀態。
14. 回放播放應依 `timeline.event_no` 或 `timeline.elapsed_ms`，不可依陣列以外的即時接收時間。

9. 錯誤代碼

code	建議前端處理
<code>bad_json</code>	顯示連線資料格式錯誤，寫 log
<code>unknown_type</code>	寫 log，可忽略
<code>unauthorized</code>	回登入頁
<code>invalid_credentials</code>	顯示帳密錯誤

code	建議前端處理
<code>username_taken</code>	顯示帳號已存在
<code>invalid_token</code>	清除 token · 回登入頁
<code>not_in_table</code>	清除桌面狀態或回大廳
<code>already_in_table</code>	顯示已在牌桌
<code>table_not_found</code>	回大廳
<code>seat_unavailable</code>	重新入桌
<code>buyin_too_low</code>	顯示最小帶入
<code>buyin_too_high</code>	顯示最大帶入
<code>insufficient_balance</code>	顯示餘額不足
<code>not_your_turn</code>	停用操作按鈕 · 等待狀態同步
<code>invalid_action</code>	顯示操作不合法 · 等待下一包 <code>table_state</code>
<code>amount_required</code>	提醒輸入金額
<code>amount_too_low</code>	顯示最低下注或加注
<code>amount_too_high</code>	顯示籌碼不足或超過上限
<code>hand_not_found</code>	顯示找不到牌局
<code>replay_forbidden</code>	顯示無權查看
<code>internal_error</code>	顯示系統忙碌 · 寫 log

10. 完整測試流程

10.1 連線與心跳

步驟：

1. 使用服務提供者給的 WebSocket URL 連線。
2. 送 `ping`。
3. 預期收到 `pong`。
4. 送錯誤 JSON。

5. 預期收到 `error code=bad_json` 或連線保持可恢復。
6. 送未知 `type`。
7. 預期收到 `error code=unknown_type`。

驗收：

- 前端不因未知訊息崩潰。
- 斷線後可重連。

10.2 登入與恢復登入

步驟：

1. 送 `guest_login` 或 `login`。
2. 預期收到 `login_ok`。
3. 保存 token。
4. 關閉連線並重連。
5. 送 `auth_token`。
6. 預期收到 `auth_ok`。

驗收：

- token 可恢復登入。
- token 失效時會回登入流程。

10.3 大廳與入桌

步驟：

1. 登入後送 `enter_lobby`。
2. 預期收到 `lobby_state`。
3. 送 `enter_game game_id=texas_holdem`。
4. 預期收到 `game_lobby_state`。
5. 送 `join_stakes`。
6. 預期收到 `table_joined`、`wallet_state`。
7. 若桌上人數足夠，預期收到 `table_countdown`。

驗收：

- 前端能顯示盲注級別。
- 前端能顯示桌上玩家與自己的座位。
- `waiting_this_hand=true` 時不可讓玩家操作。

10.4 標準一手牌

步驟：

1. 等待 `hand_start` 。
2. 收到兩包 `post_blinds` 。
3. 收到每位玩家兩張 `deal_card` 。
4. 自己收到 `deal_private` 與 `hole_cards` 。
5. 收到 `betting_start` 。
6. 輪到自己時收到 `action_request` 與 `turn` 。
7. 依 `allowed` 送 `player_action` 。
8. 觀察每次操作後收到 `player_action` 、 `table_state` 。
9. 每街結束收到 `betting_complete` 。
10. flop 、 turn 、 river 分別收到 `deal_community` 。
11. river 結束後收到 `showdown` 、 `award` 、 `hand_end` 。

驗收：

- 玩家下注、底池、目前最高下注能正確顯示。
- 換街時上一街 `bets` 歸零。
- 派彩動畫在 `award` 播放。
- 最終籌碼以 `hand_end.table.players[].chips` 顯示。

10.5 操作合法性

測試情境：

1. `to_call=0` 時應允許 `check` 、 `bet` 、 `allin` 。
2. `to_call>0` 時應允許 `fold` 、 `call` 、 `raise` 、 `allin` 。
3. 未輪到自己時送 `player_action` 。
4. `raise amount` 小於 `min_raise_to` 。
5. `amount` 超過玩家籌碼。

驗收：

- 前端只顯示 `allowed` 內的操作。
- 錯誤操作收到 `error` 後不破壞畫面。

10.6 Timeout 與斷線

步驟：

1. 輪到玩家時不操作。
2. 若可 check，預期收到 `player_action action=check_timeout` 。

3. 若不可 check，預期收到 `player_action action=fold_timeout`。
4. 測試玩家斷線後重連。

驗收：

- 前端倒數結束與服務結果一致。
- `is_disconnected` 能正確顯示。
- 重連後畫面能由 `table_state` 重建。

10.7 離桌與換桌

步驟：

1. 玩家未入手時送 `leave_room`。
2. 玩家已入手時送 `leave_room`。
3. 玩家已入手時送 `switch_room`。

驗收：

- 入手中離桌會導致自動 fold。
- 原桌收到 `table_state` 移除或更新該玩家。
- 新桌收到 `table_joined`。
- `wallet_state` 正確更新。

10.8 All fold、未跟注金額、抽水

情境：

TEXT

```
A bet 50
B raise 100
A call 100
B allin 1000
A fold
```

驗收：

- A 已投入的 100 留在底池。
- B 未被 A 跟注的多餘 all-in 金額應退回。
- 未被跟注金額不抽水。
- 若 flop 前結束，不抽水。
- `award.player_results[].net_amount` 合理。

10.9 Showdown、分池、抽水

測試情境：

1. 一般單一贏家。
2. 平分彩池。
3. 有玩家 all-in 形成 side pot。
4. 有玩家中途 fold，仍有已投入籌碼。

驗收：

- `award_breakdown[].pot_index` 可表示不同池。
- 獲勝者 `win_amount` 總和等於 `rake.payout_total`。
- 所有人 `contrib_amount` 總和等於 `rake.pot_total`。
- `rake.pot_total - rake.rake_amount = rake.payout_total`，有未跟注退回時以回傳 rake 欄位為準。

10.10 補碼

步驟：

1. 玩家籌碼低於可繼續遊戲條件。
2. 預期收到 `rebuy_offer`。
3. 送 `rebuy_decision accept=true`。
4. 預期收到 `rebuy_ack`、`rebuy_result`、`wallet_state`。
5. 測試餘額不足。
6. 預期收到 `error` 或 `rebuy_result ok=false`。

驗收：

- 補碼成功後桌上籌碼增加。
- 錢包餘額扣除。
- 補碼失敗不改變畫面籌碼。

10.11 牌局回放

步驟：

1. 完成一手牌。
2. 送 `hand_replay`。
3. 預期收到 `hand_replay_ok`。
4. 依 `timeline` 重播。
5. 檢查 fold 玩家在下一街 `bets` 不殘留。
6. 檢查 `award` 與 `hand_end` 順序。

驗收：

- 不出現三個玩家都顯示全輸但沒有贏家的畫面。
- all-fold 贏家即使牌型較弱，也應因其他人 fold 獲勝。
- 回放中 folded player 不參與 showdown 比牌。
- `players[].net_amount` 可能全部小於等於 0 的唯一合理情境是抽水大於贏家淨利，但仍必須有 `awards` 贏家。

10.12 報表

步驟：

1. 送 `my_hand_reports`。
2. 預期收到 `my_hand_reports_ok`。
3. 送 `my_daily_settlement_14d`。
4. 預期收到 `my_daily_settlement_14d_ok`。

驗收：

- 報表金額可正確顯示正負。
- 小數字串可轉為數字或格式化顯示。

10.13 Partner 流程

若前端不負責 Partner 流程，可跳過。若需要測試：

1. 送 `partner_auth`。
2. 送 `partner_create_member`。
3. 送 `partner_login_link`。
4. 送 `partner_wallet_deposit`。
5. 送 `partner_wallet_balance`。
6. 送 `partner_wallet_withdraw`。
7. 送 `partner_hand_replay`。

驗收：

- 每個 Partner request 都有對應 `_ok` response 或 `error`。
- `tx_id` 不應重複造成重複上分或下分。

11. 前端最小實作清單

前端至少需要支援：

1. WebSocket 連線、重連、心跳。

2. `login`、`guest_login`、`auth_token`。
3. `enter_lobby`、`enter_game`、`join_stakes`。
4. `table_joined`、`table_state`、`wallet_state`。
5. 發牌、下注、換街、showdown、award、hand_end。
6. `action_request` 驅動操作按鈕。
7. `error` 與 `forced_logout`。
8. `hand_replay` 與 `hand_replay_ok`。
9. 報表頁需要 `my_hand_reports` 與 `my_daily_settlement_14d`。